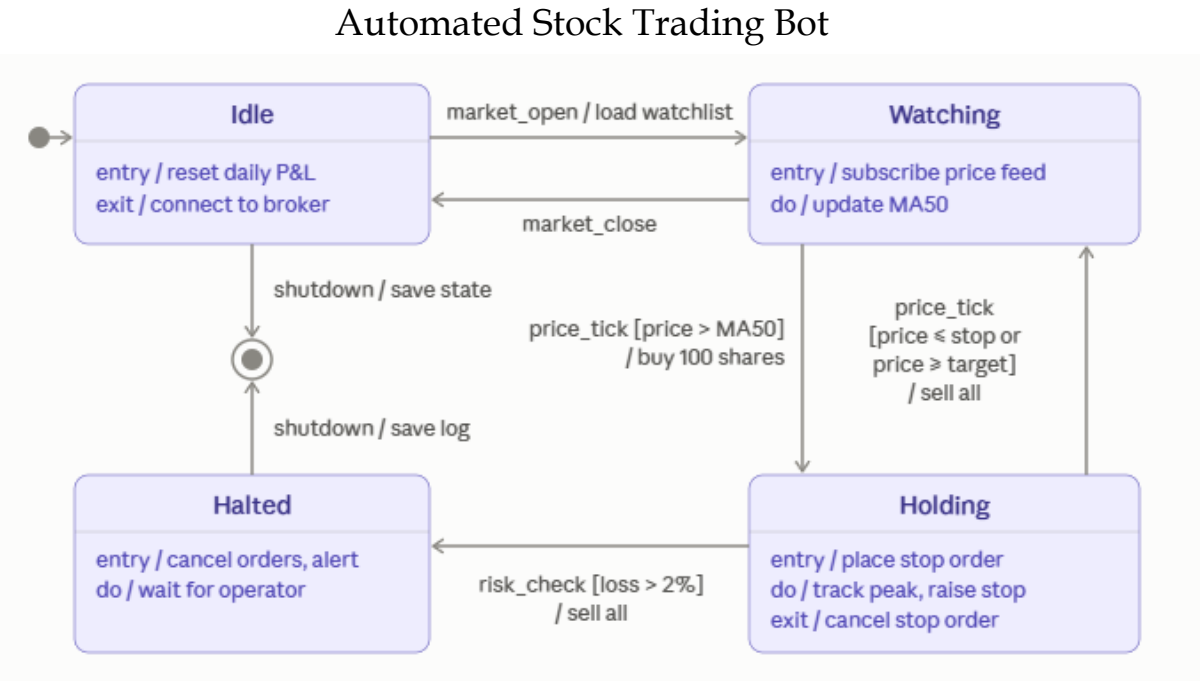
Behavior Models

State Machine Diagram

Behavioral Model – State Machine Diagram

- State machine diagram models the **finite set of states** an object or system can occupy, the **events** that cause it to move between those states, and the **conditions and actions** attached to those moves.
- At any instant the object is in exactly one state, and its response to an event depends on which state it's currently in.
- Its purpose is to describe how something behaves over its lifetime rather than how it's structured.



Behavioral Model – State Machine Diagram

- How a state machine diagram helps you?
  - It clarifies requirements by forcing you to enumerate every state and decide, for each event, what should happen.
  - It exposes invalid behavior. Any transition not drawn is, by definition, not allowed.
  - It guides implementation and testing. The diagram maps almost directly to code, and each arrow becomes a test case.
  - It communicates behavior compactly. Developers, testers, and domain experts can all read the same picture.

Use Case Description

- Scenarios in use case description represent sequence of activities that involves actors and the system
- The actor and the system exchange information in the sequence of activities
- It is one of the good source to identify
  - States
  - Events
  - Conditions
  - Actions

Identifying States

- States appear in preconditions and postconditions in status words and adjectives (“pending,” “working,” “suspended”).
- In the gaps between steps where the system has done its part and is waiting for an actor, an external system, or time to do something.
- Looks from the above two points for candidate states.

Identifying Events

- Events are whatever ends a wait: -
  - an actor’s action
  - a message from another system
  - time passing (“after 30 days,” “at market close”)
- The trigger at the top of a use case is also an event
- The conditions that start alternative flows are usually events too.

Identifying Conditions

- Conditions show up as “if,” “only if,” “provided that,” validation rules, and business-rule references.
- In alternative flows, watch for whether the branch is caused by something happening (an event) or by something being true (a guard)

Identifying Actions

- Actions are the system's responses: verbs like "records," "sends," "notifies," "reserves," "calculates."
- Once you've found them, decide where each belongs.
  - If it happens whenever the object enters or leaves a state, make it an entry or exit hook
  - if it's tied to one specific move, put it on that transition.

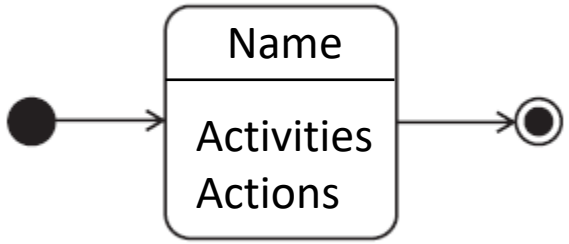
Working Example

- Scenario: borrowing a book from a library. The object being modeled is a single copy of a book.
- Precondition: member is registered and the book is on the shelf.
- Main flow:
  1. Member asks to borrow the book.
  2. System checks that the member has fewer than 5 books on loan.
  3. System records the loan and sets a due date 14 days ahead.
  4. Member returns the book by the due date.
  5. System records the return and the book goes back on the shelf.
- Alternative flows:
  2a. Member already has 5 books: system refuses the loan.
  4a. Member renews before the due date: if nobody has reserved the book, system extends the due date by 14 days.
  4b. Due date passes without a return: system emails the member a reminder.
  4c. Member returns an overdue book: system charges a late fine.
  4d. Book is 30 days overdue: system marks it lost and bills the member for a replacement.
  4e. A lost book turns up: system waives the replacement bill.
- Business rule: a librarian may withdraw a damaged book from circulation while it's on the shelf.
- Postcondition (success): the loan is closed, the book is Available on the shelf, and the member's loan count is reduced by one.

Working Example

Element	Source	Result
States	Pre-condition; waits around step 3; alternatives 4b and 4d	Available, On loan, Overdue, Lost
Events	Trigger (steps 1); Step 4; alternatives 4a–4e; business rule	borrow, return, renew, due_date_passed, after(30 days), found, withdraw
Conditions	Step 2, alternative 4a, business rule	[loans < 5], [not reserved], [damaged]
Actions	Steps 3 and 5; alternatives 4a–4e	set due date, extend due date, send reminder, charge fine, bill replacement, waive bill

Notation



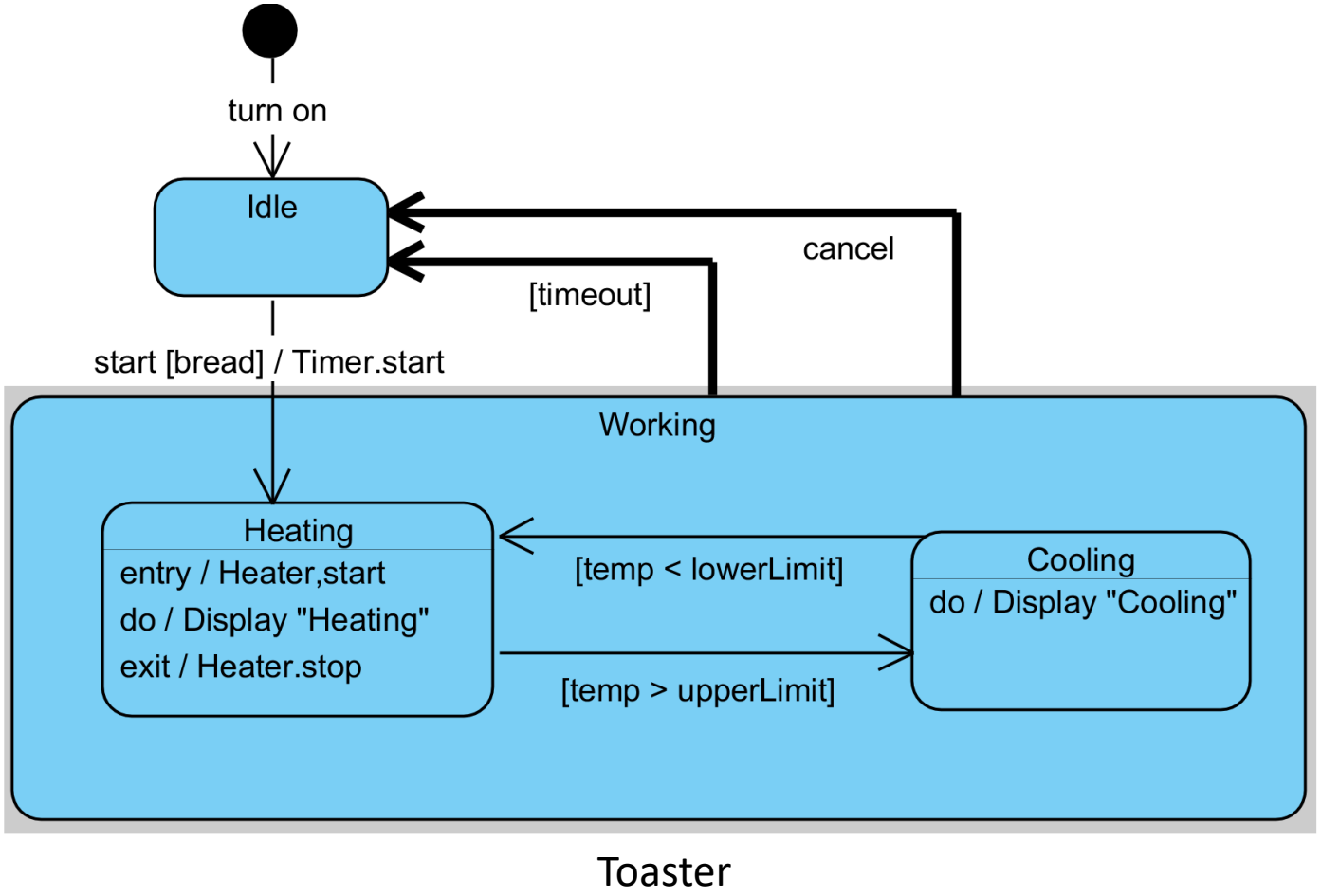
- Start and Terminate
- State
- Transition
- Actions and activities

Start / Terminate State

- Start State: Creation of object
- Terminate State: Destruction of object

State

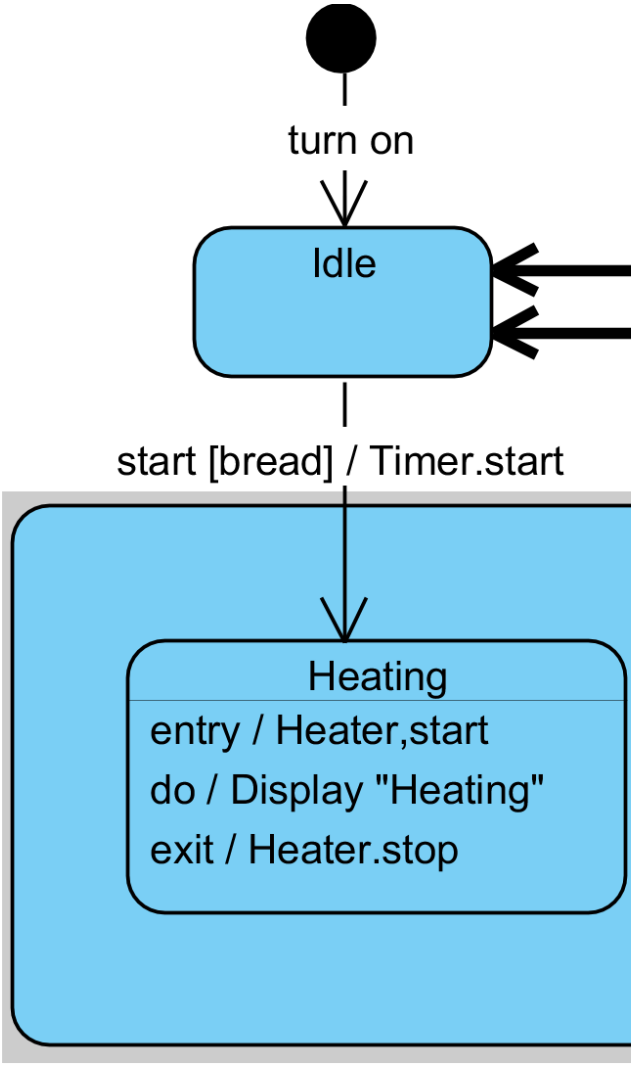
- Each class has one or more states
- Class behave differently depending on each state
- Class can change from one state to the next through Transition
- Each state must have incoming and outgoing transitions



Toaster

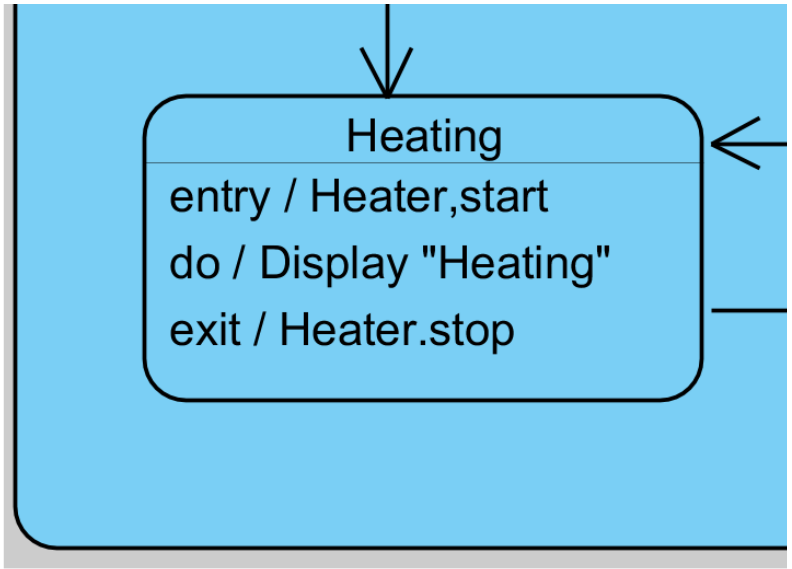
The transition

- Change from one state to the other
- What caused transition?
  - Event
  - Condition met during an event
- Actions may be executed during transition
- event[guard]/action



Action and activity

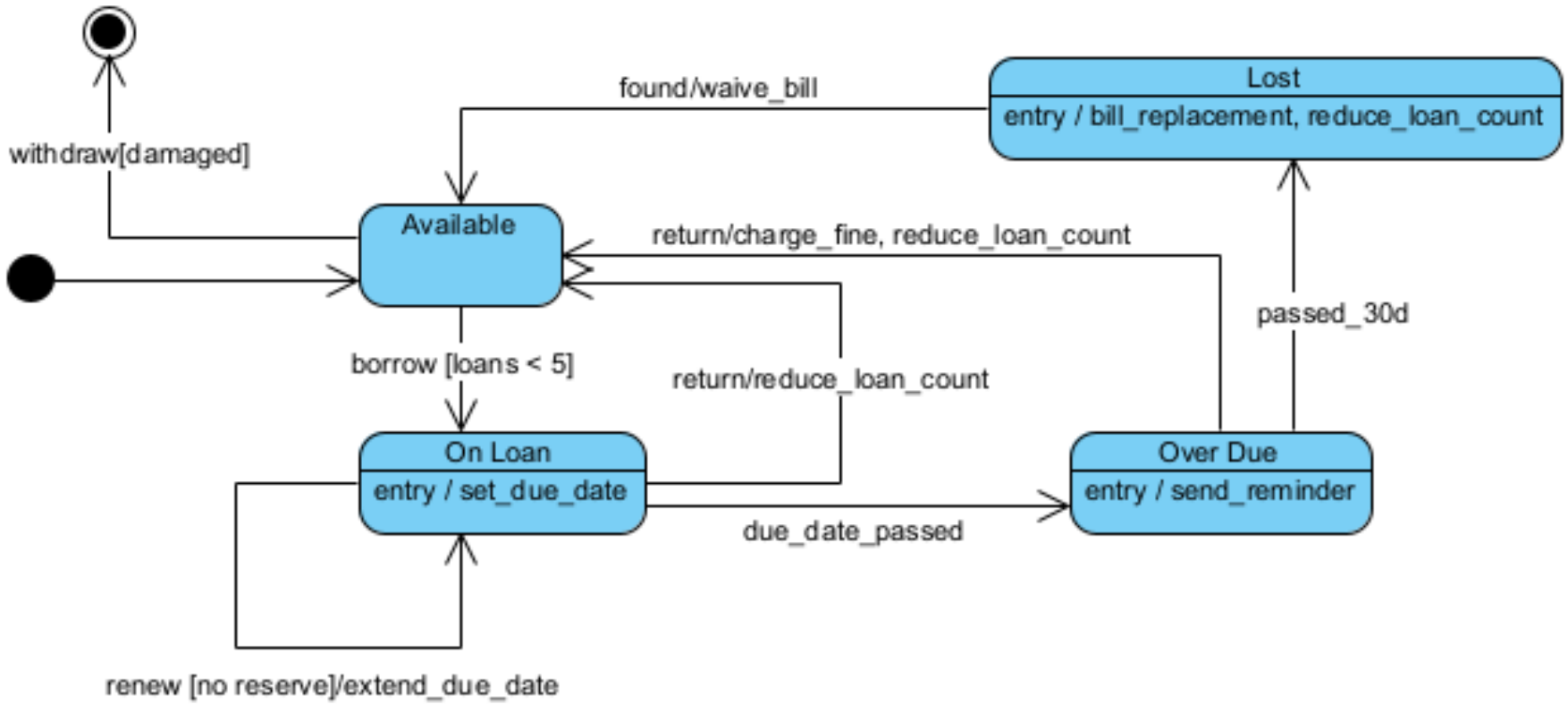
- entry/action: discrete action, perform only once at the entrance to the state
- do/activity:
  - continuous activity throughout the state live span
  - starts after the entry/action completed
  - interrupted (aborted) the moment an outgoing transition fires
  - may also finish on its own, which can trigger a completion transition out of the state
- exit/action: discrete action, perform only once at the exit of the state



Working Example

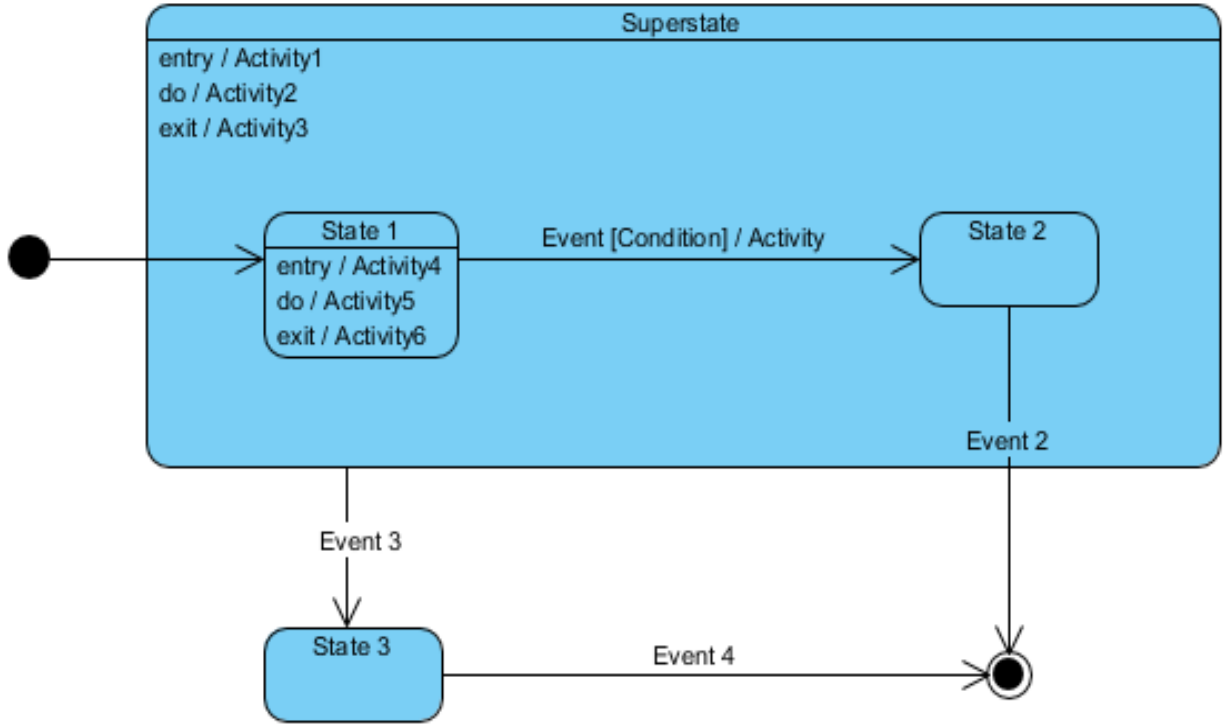
- Draw the working example with the identified elements for “Borrowing a book from a library” scenario

Working Example

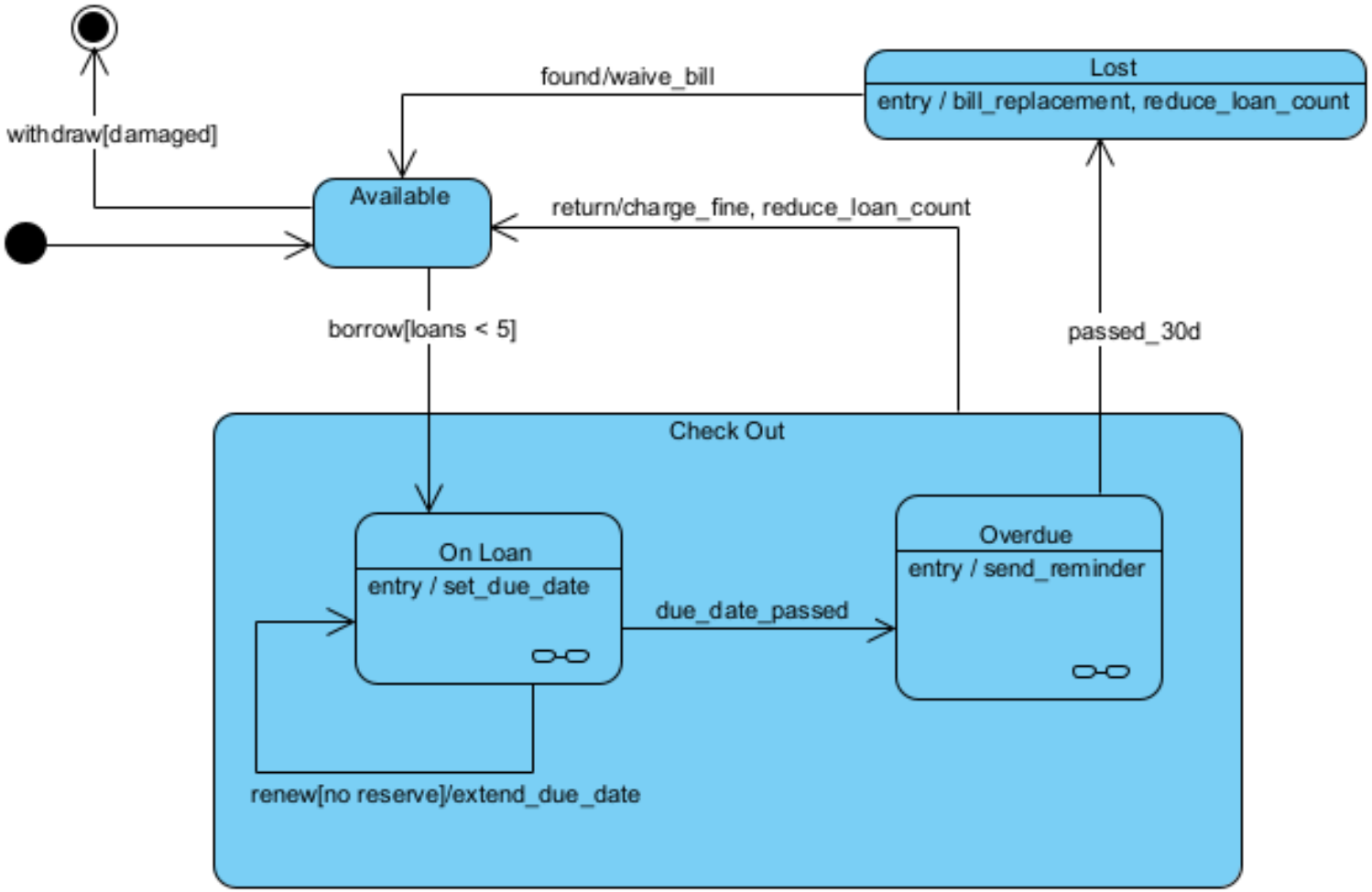


State Hierarchy

- A superstate (UML calls it a composite state) is a state that contains other states, called substates.
- When the object is in a substate, it's simultaneously in the superstate around it.
- A superstate captures behavior that several states have in common, so it's stated once at a higher level.
- Superstate/substate hierarchy lets readers view the model at different levels of detail. It also simplify complex diagram.



Working Example with Superstate

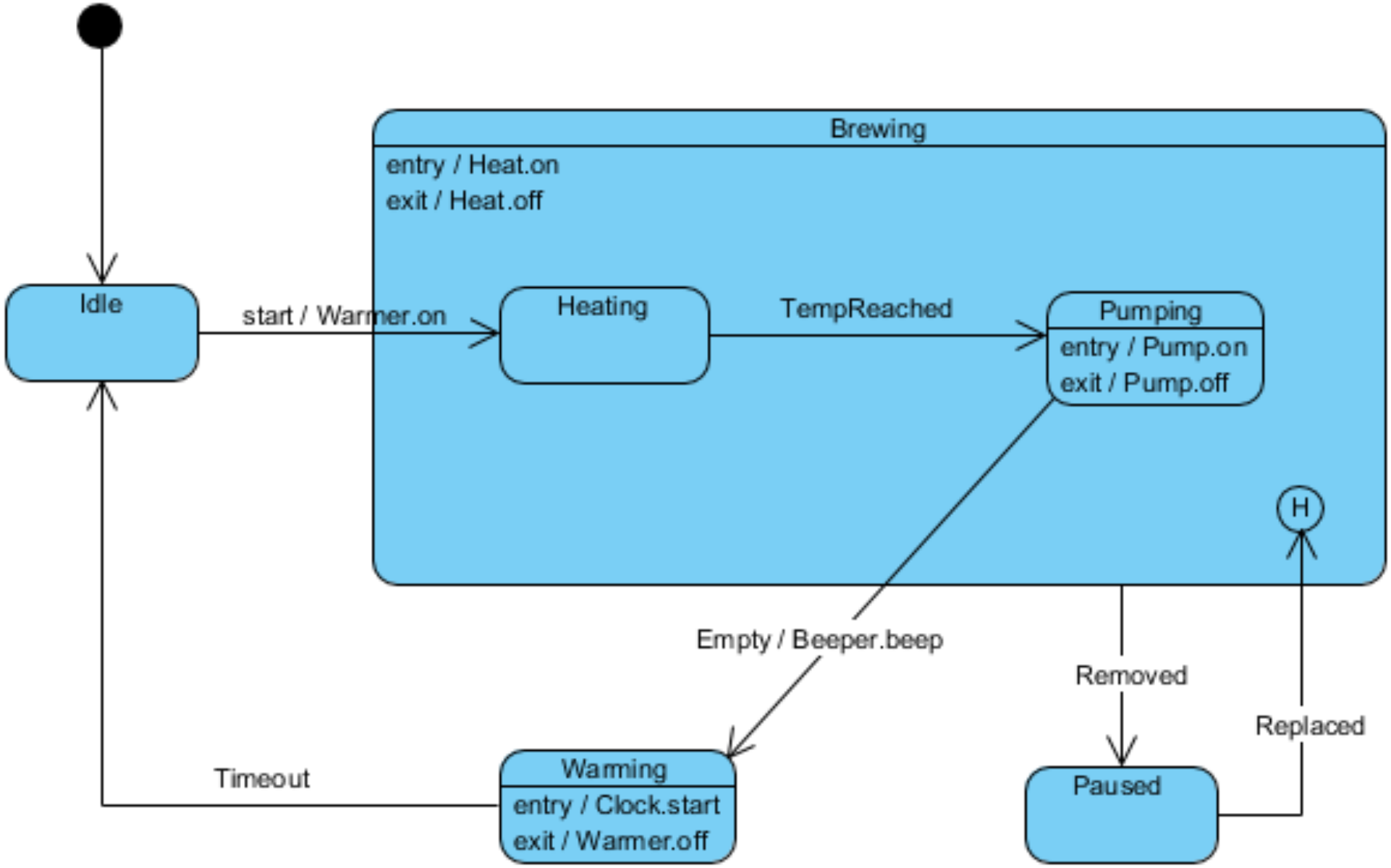


Example: Modeling an automatic coffee maker



- Person fills with coffee grounds & water filled
- Person puts carafe in place
- Person presses the Start button
- System starts brewing coffee
  - Heating element and carafe warmer turned on
  - When brewing temp reached, pump is started
  - Stop pump, heating element, and sound beeper when water level is empty
- Automated timer begins when brewing is complete
- When timer elapses, turn off carafe warmer.

Source from: Rob Pettit, 2005



- History pseudostate (H) — resumes whichever sub-state was last active

Summary

- **Transition** is labelled as event[guard]/action
- **entry/:** runs once when the object enters the state
- **do/:** runs continuously while in the state, and stops when an outgoing transition fires
- **exit/:** runs once when the object leaves the state
- A **superstate** (composite state) contains substates. While the object is in a substate, it is also in the superstate
- A **history pseudostate (H)** resumes the substate that was last active